



PALADIN
BLOCKCHAIN SECURITY

Smart Contract Security Assessment

Final Report

For Layer Zero Read

15 Nov 2024



paladinsec.co



info@paladinsec.co

Table of Contents

Table of Contents	2
Disclaimer	4
1 Overview	5
1.1 Summary	5
1.2 Contracts Assessed	6
1.3 Findings Summary	7
1.3.1 CmdLib	8
1.3.2 CmdLibBase	8
1.3.3 Multisig	8
1.3.4 DVN	8
1.3.5 CmdCodecV1	8
1.3.6 OAppRead	9
2 Findings	10
2.1 CmdLib	10
2.1.1 Privileged Functions	10
2.1.2 Issues & Recommendations	11
2.2 CmdLibBase	13
2.2.1 Privileged Functions	13
2.2.2 Issues & Recommendations	14
2.3 Multisig	15
2.3.1 Privileged Functions	15
2.3.2 Issues & Recommendations	15
2.4 DVN	16
2.4.1 Privileged Functions	16
2.4.2 Issues & Recommendations	18
2.5 CmdCodecV1	19
2.5.1 Privileged Functions	19
2.5.2 Issues & Recommendations	20
2.6 OAppRead	22
2.6.1 Privileged Functions	22



Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocacy for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team. Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

1 Overview

This report has been prepared for Layer Zero Read on the Multichain network. Paladin provides a user-centred examination of the smart contracts to look for vulnerabilities, logic errors or other issues from both an internal and external perspective.

1.1 Summary

Project Name	Layer Zero Read
URL	
Platform	Multichain
Language	Solidity
Preliminary	https://github.com/LayerZero-Labs/monorepo/tree/80d5418ab4eb1-75f1f9e65c9e7beb91dab1dfd6d/packages/layerzero-v2/evm
Resolution	https://github.com/LayerZero-Labs/monorepo/tree/56cc7e7ecbc5b-eb469d6b494e0c5e44de348e5ab/packages/layerzero-v2/evm

1.2 Contracts Assessed

Name	Contract	Live Code Match
CmdLib		PENDING
CmdLibBase		PENDING
Multisig		PENDING
DVN		PENDING
CmdCodecV1		PENDING
OAppRead		PENDING

1.3 Findings Summary

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)
● Governance	-	-	-	-
● High	-	-	-	-
● Medium	1	1	-	-
● Low	3	2	-	1
● Informational	7	1	-	6
Total	11	4	-	7

Classification of Issues

Severity	Description
● Governance	Issues under this category are where the governance or owners of the protocol have certain privileges that users need to be aware of, some of which can result in the loss of user funds if the governance's private keys are lost or if they turn malicious, for example.
● High	Exploits, vulnerabilities or errors that will certainly or probabilistically lead towards loss of funds, control, or impairment of the contract and its functions. Issues under this classification are recommended to be fixed with utmost urgency.
● Medium	Bugs or issues that may be subject to exploit, though their impact is somewhat limited. Issues under this classification are recommended to be fixed as soon as possible.
● Low	Effects are minimal in isolation and do not pose a significant danger to the project or its users. Issues under this classification are recommended to be fixed nonetheless.
● Informational	Consistency, syntax or style best practices. Generally pose a negligible level of risk, if any.

1.3.1 CmdLib

ID	Severity	Summary	Status
01	LOW	OApps can only use channel ids with a default config	✓ RESOLVED
02	INFO	Gas optimizations	ACKNOWLEDGED
03	INFO	treasuryGasLimit might be insufficient in case gas costs change in the future	ACKNOWLEDGED

1.3.2 CmdLibBase

ID	Severity	Summary	Status
04	INFO	Use Ownable2Step as a safer alternative to Ownable	ACKNOWLEDGED

1.3.3 Multisig

No issues found.

1.3.4 DVN

ID	Severity	Summary	Status
05	INFO	Insufficient validation	ACKNOWLEDGED
06	INFO	Gas optimizations	ACKNOWLEDGED

1.3.5 CmdCodecV1

ID	Severity	Summary	Status
07	INFO	The decode functions would revert with no error on invalid cmd bytes	ACKNOWLEDGED
08	INFO	Lack of safe cast	✓ RESOLVED

1.3.6 OAppRead

ID	Severity	Summary	Status
09	MEDIUM	Lack of differences between a LZ read message and a regular message	✓ RESOLVED
10	LOW	setReadChannel could overwrite previous values by mistake	ACKNOWLEDGED
11	LOW	A channel id could be assigned to an eid in the future	✓ RESOLVED

2 Findings

2.1 CmdLib

CmdLib inherits from the **CmdLibBase** contract and is used as a send and receive library on LayerZero's EnpointV2.

It contains the logic of the **SendLibrary** and **ReceiveLibrary** merged in a single contract to allow sending and receiving information. This contract is meant to be used within LayerZero Read which allows a contract to receive information from other contracts from other chains. For example, Layer Zero Read can be used to create an Oracle on chain A that aggregate the TWAP price of chain B and C.

2.1.1 Privileged Functions

- `setTreasury`
- `setTreasuryNativeFeeCap`

2.1.2 Issues & Recommendations

Issue #01	OApps can only use channel ids with a default config
Severity	 LOW SEVERITY
Description	OApps can only set their config using a channel id that have a default config. This will restrain the number of channel id they can use, which could prevent them to set different configs for all the different type of messages they wish to have. It will be especially more annoying because the documentations and comments don't specify any channel id to use, and from the OAppRead contract itself, it looks like any channel id could be used, even an id that is already used for a regular eid.
Recommendation	Consider only allowing a single channel id to be used, for example channel id 1, that way it will also be easier for OApps to know whether this message is from a lzRead or a regular message. This would also make it easier for DVNs as they would know directly which packed's src and dst should be swapped. If it is desired to allow different configurations for different type of lRead message, consider using a range of ids that will not ever be reused for an eid and that would be available on all chains (for example, 1 to 10...)
Resolution	 RESOLVED
	LayerZero plans to publish an OApp example that allows both send and read, with protections against sending to a read EID. Right now we are only sharing a send OApp and a read OApp example, but not an oapp that is dual use, which appears to be necessary for an OApp to be exposed.

Issue #02	Gas optimizations
Severity	● INFORMATIONAL
Description	- Within <code>setTreasuryNativeFeeCap()</code>, consider updating the <code>></code> operator in the following check: <pre>if (_newTreasuryNativeFeeCap > treasuryNativeFeeCap)</pre> to <code>>=</code> as it would prevent setting it to the same value. - Within <code>send()</code>, consider checking at the beginning that the <code>dstEid</code> is supported, instead of waiting until the <code>getCmdLibConfig()</code> call inside <code>_payWorkers()</code> reverts. This will save unnecessary checks and storage loads. The following check can be used: <code>assertSupportedEid(packet.dstEid)</code>
Recommendation	Consider implementing the above recommendations
Resolution	● ACKNOWLEDGED

Issue #03	treasuryGasLimit might be insufficient in case gas costs change in the future
Severity	● INFORMATIONAL
Description	The immutable <code>treasuryGasLimit</code> variable is used to determine the amount of gas used when executing a low-level call to pay fees to the treasury. It is possible that the gas cost of some opcodes changes (increases) in the future so that the hardcoded <code>treasuryGasLimit</code> might be insufficient for the execution of the low-level call to the treasury.
Recommendation	When setting the value of <code>treasuryGasLimit</code> make sure to consider the potential changes in gas cost to make sure the call would still work even in that scenario.
Resolution	● ACKNOWLEDGED

2.2 CmdLibBase

`cmdLibBase` contains the base logic for the `CmdLib` contract. It has the same logic as `UlnBase` and implement the same functionalities.

It allows the default cmd lib configuration to be set and to return an OApp configuration by using either its custom configuration or the default one if a parameter is not set on the custom one.

It also contains internal functions to help setting the OApp configurations that should be used by the contract inheriting this base contract.

2.2.1 Privileged Functions

- `setDefaultCmdLibConfigs`

2.2.2 Issues & Recommendations

Issue #04	Use Ownable2Step as a safer alternative to Ownable
Severity	● INFORMATIONAL
Description	The contract uses Ownable for access control. A safer alternative is to use Ownable2Step - rather than directly transferring to the new owner, the transfer only completes when the new owner accepts ownership.
Recommendation	Consider using Ownable2Step instead of Ownable . This makes ownership transfers much safer and prevents accidental transfer to invalid addresses.
Resolution	● ACKNOWLEDGED

2.3 Multisig

Multisig allows the setting of **signers** that can sign messages to allow certain action on specific contracts.

It also defines getter functions to allow querying information about the multisig parameters such as the signers and the quorum.

2.3.1 Privileged Functions

None.

2.3.2 Issues & Recommendations

No issues found.

2.4 DVN

DVN defines a Decentralized Verifier Network (DVN) on the Layer Zero protocol, designed to facilitate secure cross-chain operations with Layer Zero's Ultra Light Node (ULN) technology. It leverages multi-signature (MultiSig) functionality, allowing operations to be approved only when a quorum of designated signers validates actions. The contract uses an immutable, unique identifier (**vid**) for this DVN instance, helping distinguish it within the network.

Through its **Execute** function, it processes a series of operations, each with specific target addresses, call data, expiration, and required signatures. The contract allows admins to adjust destination configurations (**DstConfig**) and set gas fees through specific configuration parameters. It includes access control modifiers to restrict critical operations, allowing only the contract itself or admins to execute certain actions.

Furthermore, the DVN integrates fee management through an external fee library (- **IDVNFeeLib**), supporting flexible, custom fee calculations based on destination configurations. The contract supports Layer Zero V2 features, such as withdrawal and fee adjustment functions, which enhances secure job assignments, validation, and signature verification across interconnected blockchains.

2.4.1 Privileged Functions

- `setSigner [quorum signatures from signers]`
- `setQuorum[quorum signatures from signers]`
- `grantRole[DEFAULT_ADMIN]`
- `revokeRole[DEFAULT_ADMIN]`
- `quorumChangeAdmin[quorum signatures from signers]`
- `setDstConfig [ADMIN_ROLE]`
- `execute [ADMIN_ROLE with quorum signatures from signers]`
- `withdrawFeeFromULnV2 [ADMIN_ROLE]`
- `assignJob [MESSAGE_LIB_ROLE] and [ACL for the sender of the message]`
- `getFee [ACLfor the sender of the message]`
- `setPaused [ADMIN_ROLE]`
- `setWorkerFeeLib [ADMIN_ROLE]`
- `setDefaultMultiplierBps [ADMIN_ROLE]`

- withdrawFee [ADMIN_ROLE]
- withdrawToken [ADMIN_ROLE]
- setSupportedOptionTypes [ADMIN_ROLE]



2.4.2 Issues & Recommendations

Issue #05	Insufficient validation
Severity	● INFORMATIONAL
Description	• Within <code>setSigner()</code>, validate that the provided <code>signer</code> address cannot be <code>address(0)</code>. • Inside the constructor of the inherited <code>Worker</code> contract, check that the provided <code>_admins</code> array is not empty to ensure the contract is not locked from executing any permissioned functions. Additionally check that the <code>_priceFeed</code> argument is not <code>address(0)</code>. • Implement <code>address(0)</code> check to the following functions in <code>Worker</code> • <code>setPriceFeed()</code> • <code>setWorkerFeeLib()</code>
Recommendation	Consider implementing the above recommendations.
Resolution	● ACKNOWLEDGED

Issue #06	Gas optimizations
Severity	● INFORMATIONAL
Description	Within <code>quorumChangeAdmin()</code> , move the <code>usedHashes[hash]</code> check first and only verify the signatures after that. This way, unnecessary loops and ECDS recover calculations will be skipped which will save gas.
Recommendation	Consider implementing the above recommendation.
Resolution	● ACKNOWLEDGED

2.5 CmdCodecV1

The `CmdCodecV1` library is a Solidity library for encoding and decoding structured EVM read requests. It defines two main structures `EVMCallRequestV1` and `EVMCallComputeV1` to specify blockchain targets, conditions, and request parameters.

The library includes functions to:

- **Decode** a command payload into a structured format.
- **Encode** a structured request back into a command payload.
- **Append** requests and compute instructions, allowing for modular and flexible construction of cross-chain command sets.

It incorporates version control and error handling to support protocol adaptability and secure interactions.

2.5.1 Privileged Functions

None.

2.5.2 Issues & Recommendations

Issue #07	The decode functions would revert with no error on invalid cmd bytes
Severity	● INFORMATIONAL
Description	L 36 - 44 <pre>function decode(bytes calldata _cmd) internal pure returns (uint16 appCmdLabel, EVMCallRequestV1[] memory evmCallRequests, EVMCallComputeV1 memory compute) { uint256 offset = 0; uint16 cmdVersion = uint16(bytes2(_cmd[offset:offset + 2])); ... }</pre> The <code>decode</code> function does not check that <code>cmd</code> is at least 2 bytes before trying to decode <code>cmdVersion</code>. Similarly, this could also happen in the other <code>decodeX</code> function as the length is never checked to be at least greater than the required size.
Recommendation	Consider adding a check that the length of <code>cmd</code> is large enough, and if not, revert with a custom error. This would make debugging easier for builders using Layer Zero Read.
Resolution	● ACKNOWLEDGED

Issue #08	Lack of safe cast
Severity	● INFORMATIONAL
Description	The <code>calldata</code> and <code>evmCallRequest</code> length are casted to <code>uint16</code> without any checks to ensure that the result does not overflow. L 167 <pre>bytes memory cmd = abi.encodePacked(CMD_VERSION, _appCmdLabel, uint16(_evmCallRequests.length));</pre> L 188 <pre>uint16(_request.callData.length + 35)</pre>
Recommendation	Consider casting the values to <code>uint16</code> safely to prevent any issues.
Resolution	✓ RESOLVED The safe cast was added.

2.6 OAppRead

OAppRead is a helper contract designed to simplify **OApp** interactions with **LZRead**, enabling efficient querying and processing of cross-chain information on-chain.

In **LZRead** messages, channel IDs are used instead of endpoint IDs, and **CmdLib** replaces **SendLib/ReceiveLib** for message verification. Builders should note that when an **LZRead** message is sent, the received message will reference the original channel ID as the **srcEid**.

2.6.1 Privileged Functions

- **setReadChannel**
- **lzReceive [ENDPOINT]**
- **setPeer**
- **setDelegate**

2.6.2 Issues & Recommendations

Issue #09 Lack of differences between a LZ read message and a regular message

Severity

● MEDIUM SEVERITY

Description

Both LZ read messages and regular LZ messages are protected using the same logic with peers. A peer is an OApp on another chain that is allowed to interact with an OApp through Layer Zero messages. However, currently as a LZ read message and a regular LZ message lacks differences, users can abuse this to send read messages that would then be executed as real regular messages.

For this example, we'll take the example of an OFT that wants to allow reading information from other chain. The OFT has a peer on chain A and B, and has set channel id C for its LZ Read messages. The OApp have the same address on chain A and B.

Let's say that the `triggerRead` function is as such:

```
function triggerRead(
    uint32 cid,
    EVMCallRequestV1[] memory evmCallRequests,
    EVMCallComputeV1 memory evmCallCompute,
    bytes memory options
) public payable {
    MessagingParams memory msgParams = MessagingParams(
        cid,
        bytes32(uint256(uint160(address(this)))),
        CmdCodecV1.encode(1, evmCallRequests, evmCallCompute),
        options,
        false
    );
    endpointV2.send{ value: msg.value }(msgParams,
    msg.sender);
}
```

This `trigger` function can be used to mint OFT tokens to a carefully crafted address:

```
uint256 maliciousUserKey = 137; // 137 is the first key
that works
address maliciousUser = vm.addr(maliciousUserKey);

while (
```

```

        uint8(uint160(maliciousUser) >> (160 - 8 - 32 - 8))
    > 1
        || uint8(uint160(maliciousUser) >> (160 - 8)) < 35
    ) {
        maliciousUserKey += 1;
        maliciousUser = vm.addr(maliciousUserKey);
    }

    uint64 amount = 1e18;

    EVMCallRequestV1[] memory evmCallRequests = new EVMCall-
    RequestV1[](1);
    evmCallRequests[0] = EVMCallRequestV1({
        appRequestLabel: 0,
        targetEid: uint32(uint160(maliciousUser) >> (160 - 8
    - 32)),
        isBlockNum: uint8(uint160(maliciousUser) >> (160 - 8
    - 32 - 8)) == 1,
        blockNumOrTimestamp: uint64(uint160(maliciousUser) >>
    (160 - 8 - 32 - 8 - 64)),
        confirmations: uint16(uint160(maliciousUser) >> (160
    - 8 - 32 - 8 - 64 - 16)),
        to: address(uint160(maliciousUser) << (160 - 32) |
    (uint160(amount) << (160 - 32 - 64))),
        callData: new bytes(uint8(uint160(maliciousUser) >>
    (160 - 8)) - 35)
    });

    EVMCallComputeV1 memory evmCallCompute; // Empty

```

Then the attacker would simply have to call `triggerRead(eidChainB, evmCallRequests, evmCallCompute)` on chain A, and this would mint him 1e30 OFT Tokens on the chain B.

Furthermore, nothing really prevents users of an OApp to send a real message to a channel id instead of a dst id. Most of the time, it would revert due to the `CmdLib` trying to decode the data, but in some cases, this check could pass and the user might lose funds while doing so.

Recommendation

This issue can be fixed in different ways:

- Only allow OApp to be of a single type, either a regular OApp or a OAppRead. This would be the safest way, but it cannot really be enforced outside of explaining in comments and documentation that

users should not allow both actions from a single OApp (and instead use two OApps, one to read, one to send messages)

- Force the channel id and eid to be different, and always expect channel ids to be from an invalid range for eid (for example 1 to 10). Safety checks should be added to `_lzReceive` and `_lzSend` to allow for safer integration. For example, by adding a `_lzReadReceive` and `_lzReceive`, `_lzRead` and `_lzSend` that would each check that the id is from the valid range.

Resolution

✓ RESOLVED

LayerZero plans to publish an OApp example that allows both send and read, with protections against sending to a read EID. Right now we are only sharing a send OApp and a read OApp example, but not an oapp that is dual use, which appears to be necessary for an OApp to be exposed.

Issue #10

setReadChannel could overwrite previous values by mistake

Severity

● LOW SEVERITY

Description

The `setReadChannel` is used to set which channel id to use when using a LZ read message. However, as it uses the exact same logic as for the regular messages, an OApp operator could overwrite the correct peer if he uses a `channelId` that he already uses for an Eid.

This could prevent user from bridging funds, and in some cases, for example with OFT, could burn their tokens forever.

Recommendation

Consider forcing channel ids to be of a certain type (for example, from 1 to 10) and add check on both `setReadChannel` and `setPeer` that they are both on the valid range to prevent misconfigurations.

Resolution

● ACKNOWLEDGED

LayerZero plans to publish an OApp example that allows both send and read, with protections against sending to a read EID. Right now we are only sharing a send OApp and a read OApp example, but not an oapp that is dual use, which appears to be necessary for an OApp to be exposed.

Issue #11	A channel id could be assigned to an eid in the future
Severity	● LOW SEVERITY
Description	Channel ids and endpoint ids should always be different to prevent issues with messages. Even DVNs would have a hard time knowing if they should flip srcEid and dstEid , if there is ever a collision.
Recommendation	Consider setting channel ids carefully to make sure that no endpoint id can be given using an already used channel id. For example, channel ids could be given from 1 to 10, and endpoint ids would be given ids outside of this range.
Resolution	✓ RESOLVED LayerZero will carefully select read ids to never overwrite any other ids.